

42SEOUL

gshim, seongyle, jimin, yena

: 수요지식회

지난 몇개월간 집현전 개발팀의 장기프로젝트
였던 TypeORM 프레임워크 도입의 과정을
소개합니다.

좌충우돌 Raw Query 없애기 프로젝트

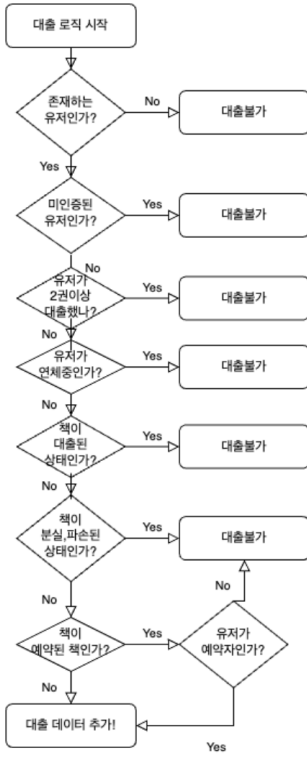
대청소의 필요성

집현전의 기능을 살펴봅시다

|

01

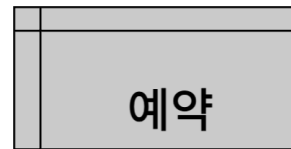
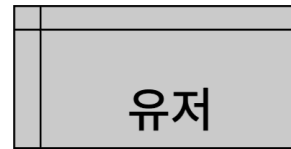
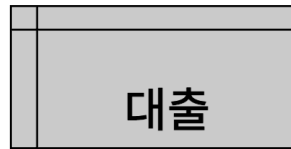
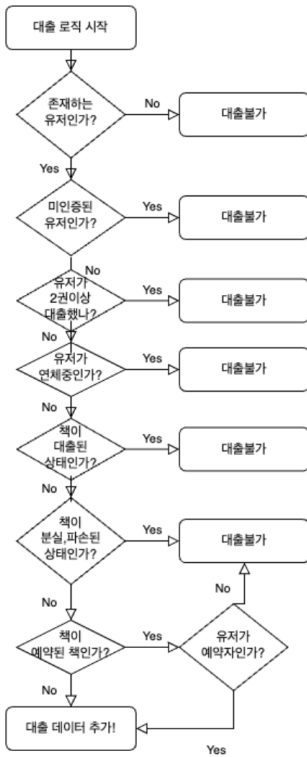
각 조건을 체크하기 위해서는?



**대출
책**

**유저
예약**

- 대출을 할 때, 대출 가능 조건을 보기 위해서는 대출 데이터만 조회하는 것이 아니라, 이 유저가 존재하는 유저인지, 대출 가능한 책인지, 예약이 되어있는지 등을 봐야 한다.



- 따라서 대출 작업은 대출 테이블, 유저 테이블, 책 테이블, 예약 테이블에 접근한다.

구 집현전 코드 개노답 3형제

대출로직함수

- 위의 작업을 수행하는 대출 로직 함수를 포함하여 세 가지의 예시가 있다. 첫번째로 살펴볼 것은 대출 로직이다.

```

18 // lending.service.ts 대출 로직
19 export const create = async (
20   --userId: number,
21   --bookId: number,
22   --librarianId: number,
23   --condition: string,
24 ): Promise<void> => {
25   --const conn = await pool.getConnection();
26   --const transactionExecuteQuery = makeExecuteQuery(conn);
27   --const dueDate = new Date(Date.now() + 14 * 24 * 60 * 60 * 1000);
28
29   --try {
30     --await conn.beginTransaction();
31     --// 존재하는 유저인지 확인
32     --const users = await transactionExecuteQuery(`
33     --SELECT *
34     --FROM user
35     --WHERE id = ?
36     --`, [userId]);
37
38     --if (!users.length) { throw new Error(errorCode.NO_USER_ID); }
39
40     --// 유저 권한 없음
41     --if (users[0].role === 0) { throw new Error(errorCode.NO_PERMISSION); }
42
43     --// 유저가 2권 이상 대출
44     --const number0fLendings = await transactionExecuteQuery(`
45     --SELECT COUNT(*) as count
46     --FROM lending
47     --WHERE userId = ? AND returnedAt IS NULL;
48     --`, [userId]);
49
50     --if (number0fLendings[0].count >= 2) { throw new Error(errorCode.LENDING_OVERLOAD); }
51
52     --// 유저가 연체중 (패널티를 받았거나 대출중인 책이 반납기한을 넘겼을때)
53     --const hasPenalty = await transactionExecuteQuery(`

```

```

52     --// 유저가 연체중 (패널티를 받았거나 대출중인 책이 반납기한을 넘겼을때)
53     --const hasPenalty = await transactionExecuteQuery(`
54     --SELECT penaltyEndDate
55     --FROM user
56     --WHERE id = ?
57     --`, [userId]);
58
59     --const isOverdue = await transactionExecuteQuery(`
60     --SELECT
61     --CASE WHEN 14 >= DATEDIFF(NOW(), DATE(createdAt)) THEN 0
62     --ELSE DATEDIFF(NOW(), DATE(createdAt))
63     --END AS overdue
64     --FROM lending
65     --WHERE userId = ? AND returnedAt IS NULL
66     --`, [userId]);
67
68     --if (hasPenalty[0].penaltyEndDate >= new Date()
69     --|| isOverdue[0].overdue) { throw new Error(errorCode.LENDING_OVERDUE); }
70
71     --// 책이 대출되지 않은 상태인지
72     --const isNotLended = await transactionExecuteQuery(`
73     --SELECT *
74     --FROM lending
75     --WHERE bookId = ? AND returnedAt IS NULL
76     --`, [bookId]);
77
78     --if (isNotLended.length !== 0) { throw new Error(errorCode.ON_LENDING); }
79
80     --// 책이 분실, 파손이 아닌지
81     --const isLendableBook = await transactionExecuteQuery(`
82     --SELECT *
83     --FROM book
84     --WHERE id = ?
85     --`, [bookId]);
86
87     --if (isLendableBook[0].status === 1) {
88     --throw new Error(errorCode.DAMAGED_BOOK);
89     --} else if (isLendableBook[0].status === 2) {
90     --throw new Error(errorCode.LOST_BOOK);
91     --}
92
93     --// 예약된 책이 아닌지

```

```

93  --// 예약된 책이 아닌지
94  --const isNotReservedBook = await transactionExecuteQuery(`
95  --SELECT *
96  --FROM reservation
97  --WHERE bookId = ? AND status = 0
98  --`, [bookId]);
99
100  --if (isNotReservedBook.length && isNotReservedBook[0].userId !== userId) {
101  --throw new Error(errorCode.ON_RESERVATION);
102  --}
103
104  --await transactionExecuteQuery(`
105  --INSERT INTO lending (userId, bookId, lendingLibrarianId, lendingCondition)
106  --VALUES (?, ?, ?, ?)
107  --`, [userId, bookId, librarianId, condition]);
108
109  --// 예약 대출 시 상태값 reservation status 0 -> 1 변경
110  --if (isNotReservedBook.length) {
111  --await transactionExecuteQuery(`
112  --UPDATE reservation
113  --SET status = 1
114  --WHERE id = ?
115  --`, [isNotReservedBook[0].id]);
116  --}
117
118  --const books: [{title: string}] = await transactionExecuteQuery(`
119  --SELECT
120  --title
121  --FROM
122  --book_info
123  --LEFT JOIN book ON
124  --book.infoId = book_info.id
125  --WHERE
126  --book.id = ?
127  --`, [bookId]);
128  --await conn.commit();
129  --const dueDate = new Date(Date.now() + 14 * 24 * 60 * 60 * 1000);
130  --publishMessage(users[0].slack, `:jiphyeonjeon: 대출 알림 :jiphyeonjeon: \n대출 하신
131  --} catch (e) {
132  --await conn.rollback();
133  --if (e instanceof Error) {

```

- 이 코드는 대출할 때 실제로 사용한 코드로, raw query가 서비스 단에 그대로 작성되어 있다.

구 집현전 코드 개노답 3형제

리뷰 검색 조건처리로직

- 그 다음으로는 리뷰 검색을 처리하는 로직이 있다.

```

33 export const getReviewsPage = async (
34   titleOrNickname :string,
35   disabled: number,
36   page: number,
37   sort: 'asc' | 'desc',
38   limit: number
39 ) => {
40   const titleOrNicknameQuery = titleOrNickname === '' ? '' : `AND (book_info.title LIKE '${titleOrNickname}'
41     OR user.nickname LIKE '${titleOrNickname}')`;
42   const disabledQuery = disabled === -1 ? '' : `AND reviews.disabled = ${disabled}`;
43   const sortQuery = `ORDER BY reviews.id ${sort}`;
44   const limitQuery = (Number.isNaN(limit)) ? `LIMIT 10` : `LIMIT ${limit}`;
45
46   const reviews = await executeQuery(`
47     SELECT
48       reviews.id as reviewsId,
49       reviews.userId as reviewerId,
50       reviews.bookInfoId,
51       reviews.content,
52       reviews.createdAt,
53       reviews.disabled,
54       book_info.title,
55       user.nickname,
56       user.intraId
57     FROM reviews
58     JOIN user ON user.id = reviews.userId
59     JOIN book_info ON reviews.bookInfoId = book_info.id
60     WHERE reviews.isDeleted = false
61       ${titleOrNicknameQuery}
62       ${disabledQuery}
63       ${sortQuery}
64       ${limitQuery}
65     OFFSET ?
66     `, [page * 10]);
67   return (reviews);
68 };

```

WHERE 절은 검색 조건에 따라 구성되는데, 복잡도가 높고 가독성이 떨어진다.

구 집현전 코드 개노답 3형제

N+1 problem

- 마지막으로 볼 것은 N+1의 문제를 안고 있던 코드이다.

```

88 // eslint-disable-next-line max-len
89 export const searchUserBynicknameOrEmail = async (nicknameOrEmail: string, limit: number, page: number) => {
90   let items = (await executeQuery(
91     `
92     SELECT
93     id, email, nickname, intraId, slack, DATE_FORMAT(penaltyEndDate, "%Y-%m-%d") as penaltyEndDate, role
94     FROM user
95     WHERE (nickname LIKE ? or email LIKE ?)
96     LIMIT ?
97     OFFSET ?;
98     `,
99     [`${nicknameOrEmail}`, `${nicknameOrEmail}`, limit, limit * page],
100    )) as models.User[];
101   items = await setOverDueDay(items);
102   const total = (await executeQuery(
103     `SELECT COUNT(*) as totalItems FROM user WHERE (nickname LIKE ? or email LIKE ?);
104     `, [`${nicknameOrEmail}`, `${nicknameOrEmail}`]));
105   const meta: types.Meta = {
106     totalItems: total[0].totalItems,
107     itemCount: items.length,
108     itemsPerPage: limit,
109     totalPages: Math.ceil(total[0].totalItems / limit),
110     currentPage: page + 1,
111   };
112   return { items, meta };
113 };

```

- `searchUserBynicknameOrEmail` 함수에서 `setOverDueDay`를 호출한다.

```

44 export const setOverDueDay = async (items: any) => {
45   const lending = await getLending();
46   if (items) {
47     return items.map((item: models.User) => {
48       const rtnObj: models.User = Object.assign(item);
49       rtnObj.lendings = lending.items.filter((lend) => lend.userId === item.id);
50       rtnObj.overDueDay = 0;
51       if (rtnObj.lendings.length) {
52         const nowDate = new Date();
53         rtnObj.lendings.forEach((lend: models.Lending) => {
54           if (lend.duedate < nowDate) {
55             rtnObj.overDueDay += Math.floor(nowDate.getTime() / (1000 * 3600 * 24)
56               - lend.duedate.getTime() / (1000 * 3600 * 24));
57           }
58         });
59       }
60       return rtnObj;
61     });
62   }
63   return items;
64 };

```

- `setOverDueDay` 함수를 보면 `getLending` 함수를 호출하여 우선은 모든 대출 데이터를 가지고 와서 그 대출 데이터 중, 사용자의 id가 화면에 표시된 것을 필터링하여 데이터 작업을 한다.

```

6  export const getLending = async () => {
7    const items = await executeQuery(`
8      SELECT
9        l.userId as userId,
10       l.createdAt as lendDate,
11       l.lendingCondition as lendingCondition,
12       bi.id as bookInfoId,
13       bi.title as title,
14       DATE_ADD(l.createdAt, INTERVAL 14 DAY) as dueDate,
15       bi.image as image,
16       bi.author as author
17     FROM lending as l
18     LEFT JOIN book as b
19       on l.bookId = b.id
20     LEFT JOIN book_info as bi
21       on b.infoId = bi.id
22     where l.returnedAt is null;
23   `) as models.Lending[];
24   await Promise.all(items.map(async (item: models.Lending, idx) => {
25     const rtnObj: models.Lending = Object.assign(item);
26     const reservedUserArr: any[] = await executeQuery(`
27       SELECT
28         *
29       FROM reservation
30       where bookInfoId = ?;
31     `, [items[idx].bookInfoId]);
32     rtnObj.overDueDay = 0;
33     rtnObj.reservedNum = reservedUserArr.length;
34     const nowDate = new Date();
35     if (rtnObj.dueDate < nowDate) {
36       rtnObj.overDueDay += Math.floor(nowDate.getTime() / (1000 * 3600 * 24))
37         - rtnObj.dueDate.getTime() / (1000 * 3600 * 24);
38     }
39     return rtnObj;
40   }));
41   return { items };
42 };

```

- `getLending` 함수는 반납하지 않은 대출 데이터를 가져온다. 검색어 한 글자 한 글자 입력할 때마다 모든 유저의 데이터를 가져와 `map` 함수를 통해 화면에 보이는 유저에 대한 작업을 하였다. 따라서 한 글자 마다 약 760 여 개의 데이터를 조회한다.

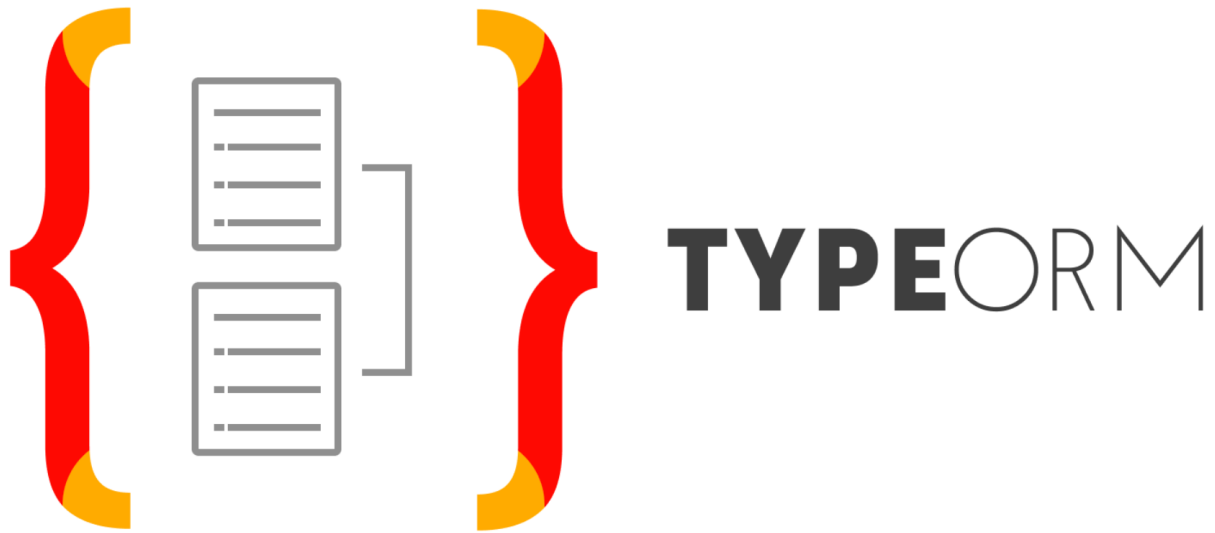
무엇이 문제였나?

비즈니스 로직과 DB접근 로직을 분리되지 않음

매번 SQL문을 작성해 필요한 데이터를 가져왔음

비효율적인 로직이 존재했음

- 요약하자면, 비즈니스 로직과 DB접근 로직을 분리하지 않은 것, 매번 SQL문을 작성하여 필요한 데이터를 가져온 것, 비효율적인 로직이 문제였다.



- 이를 해결하고자 TypeORM을 도입하였다.

왜 typeORM인가?

비즈니스 Layer와 DB Layer가 분리됩니다.

SQL 노가다지옥에서 탈출합니다.

패러다임의 한계를 극복합니다.

- TypeORM을 사용하면 비즈니스 Layer와 DB Layer가 분리되고, raw sql을 사용하지 않아도 되며, 패러다임의 한계를 극복할 수 있다는 장점이 있다.

변환 후 달라진 점들

유지보수적 이점, 성능상 이점에 대하여...!

| 02

- TypeORM 도입 후 달라진 점은 다음과 같다.

유지보수적 이점

Repository, Service Layer로 분리됨
entity CRUD 작업이 매우 간단해짐

성능상 이점

데이터 캐싱
쓰기 지연
지연로딩(lazy-loading)
변경감지

- Repository 패턴을 도입하여 Repository, Service Layer로 분리됨에 따라 CRUD 작업이 간단해졌다.
- 데이터 캐싱과 쓰기 지연, 지연 로딩, 변경 감지로 인해 성능상 이점을 취할 수 있었다.

유지보수적 이점

```
5+ usages  nyj001012 +6 *
12 export const create = async (
13   userId: number,
14   bookId: number,
15   librarianId: number,
16   condition: string,
17 ) => {
18   const dueDate = new Date( value: Date.now() + 14 * 24 * 60 * 60 * 1000);
19   const transaction = jipDataSource.createQueryRunner();
20   const lendingRepo = new LendingRepository(transaction);
21   const usersRepository = new UsersRepository(transaction);
22   try {
23     await transaction.startTransaction();
24     const [users, count] = await usersRepository.searchUserBy( conditions: { id: userId }, limit: 0, page: 0);
25     if (!count) { throw new Error(errorCode.NO_USER_ID); }
26     if (users[0].role === 0) { throw new Error(errorCode.NO_PERMISSION); }
27     // user conditions
28     const numberOfLendings = await lendingRepo.searchLendingCount( conditions: {
29       userId,
30     }, limit: 0, page: 0);
31     if (numberOfLendings >= 2) { throw new Error(errorCode.LENDING_OVERLOAD); }
32     const penaltyEndDate = await lendingRepo.getUsersPenalty(userId);
33     const overDueDay = await lendingRepo.getUsersOverDueDay(userId);
34     if (penaltyEndDate >= new Date())
35       || overDueDay !== undefined { throw new Error(errorCode.LENDING_OVERDUE); }
36     // book conditions
37     const countOfBookInLending = await lendingRepo.getLendingCountByBookId(bookId);
38     if (countOfBookInLending !== 0) { throw new Error(errorCode.ON_LENDING); }
39     // 책이 분실, 파손이 아닌지
```

```

39 // 책이 분실, 파손이 아닌지
40 const book = await lendingRepo.searchBookForLending(bookId);
41 if (book?.status === 1) {
42   throw new Error(errorCode.DAMAGED_BOOK);
43 } else if (book?.status === 2) {
44   throw new Error(errorCode.LOST_BOOK);
45 }
46 // 예약된 책이 아닌지
47 const reservationOfBook = await lendingRepo.searchReservationByBookId(bookId);
48 if (reservationOfBook && reservationOfBook.userId !== userId) {
49   throw new Error(errorCode.ON_RESERVATION);
50 }
51 // 책 대출 정보 insert
52 await lendingRepo.createLending(userId, bookId, librarianId, condition);
53 // 예약 대출 시 상태값 reservation status 0 -> 1 변경
54 if (reservationOfBook) { await lendingRepo.updateReservationToLended(reservationOfBook.id); }
55 await transaction.commitTransaction();
56 if (users[0].slack) {
57   await publishMessage(users[0].slack, msg: `:jiphyeonjeon: 대출 알림 :jiphyeonjeon: \n대출 하신 \n`);
58 }
59 } catch (e) {
60   await transaction.rollbackTransaction();
61   if (e instanceof Error) {
62     throw e;
63   }
64 } finally {
65   await transaction.release();
66 }
67 return ({ dueDate: formatDate(dueDate) });
68 };

```

- 이는 맨 처음 대출 데이터를 생성하는 함수를 다음과 같이 변환한 것이다. 한 눈에 봐도 상당히 많은 라인 수가 줄어든 것을 볼 수 있다.

```

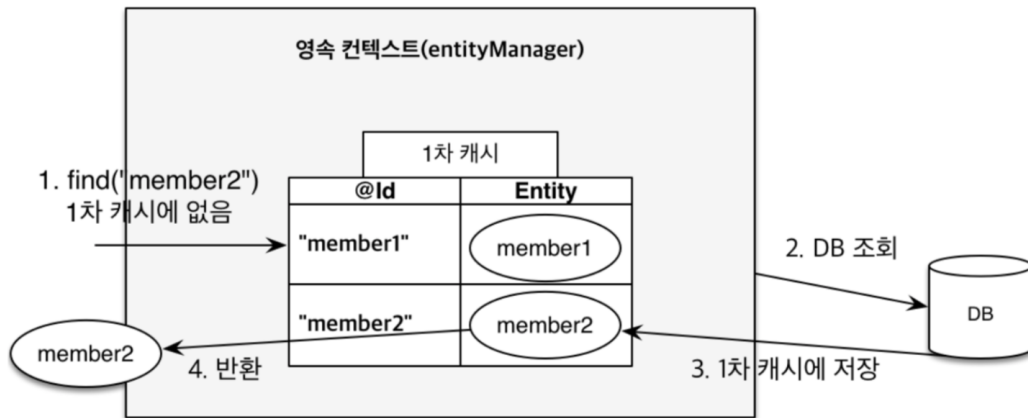
43
44 2 usages  nyj001012+2
45
46 async searchUserBynicknameOrEmail(nicknameOrEmail: string, limit: number, page: number) {
47   const [items, count] = await this.usersRepository.searchUserBy(conditions: [
48     { nickname: Like(value: `${nicknameOrEmail}%`) },
49     { email: Like(value: `${nicknameOrEmail}`) },
50   ], limit, page);
51   const setItems = await this.setOverDueDay(items);
52   const meta: types.Meta = {
53     totalItems: count,
54     itemCount: setItems.length,
55     itemsPerPage: limit,
56     totalPages: Math.ceil(x: count / limit),
57     currentPage: page + 1,
58   };
59   return { items: setItems, meta };
60 }

```

- 이전에 한 번에 760 여 개의 데이터를 가져오던 함수도 다음과 같이 리팩터링 하였다.

성능상 이점

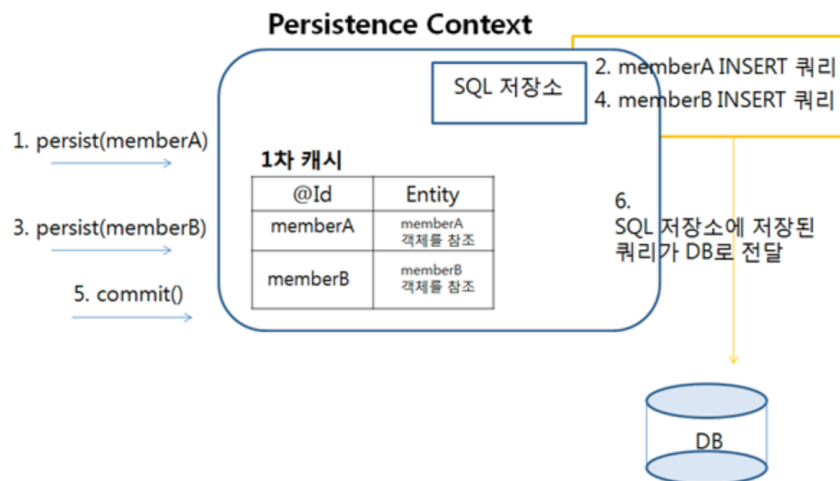
데이터 캐싱



- 유저 데이터를 테이블에서 검색을 한다는 상황을 가정해보자. 한 번이라도 조회한 데이터는 Entity Manager의 1차 캐시에 저장되고, 그 데이터를 한 번 더 조회할 때 캐시를 먼저 조회하여 캐시 데이터를 반환한다.

성능상 이점

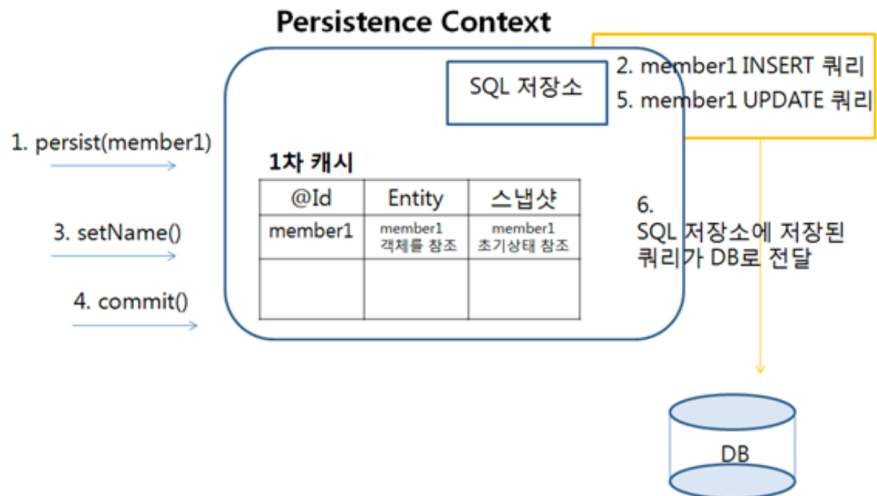
쓰기 지연



- 유저 객체를 생성한다 가정해보자. A와 B를 생성하고 커밋을 했을 때, Entity Manager에 변경 내용을 저장한 후, 커밋할 때 바뀐 내역을 일괄적으로 DB에 반영한다. 매 작업마다 DB에 요청을 보내지 않기 때문에 효율적이다.

성능상 이점

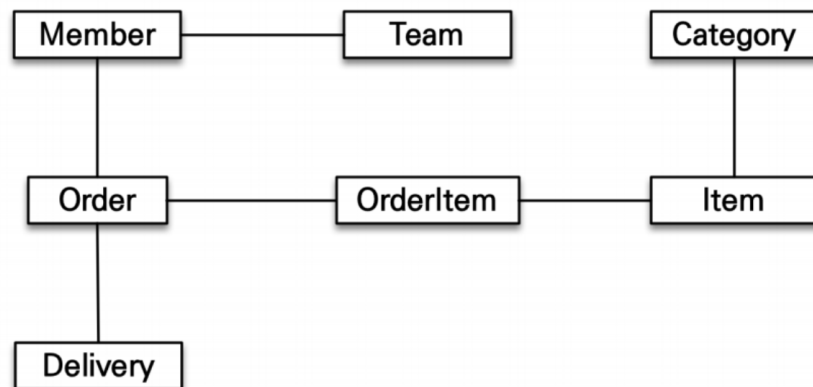
변경감지



- user 테이블에서 member1의 데이터만 업데이트 한다 가정해보자. 1차 캐시의 스냅샷에 member1이 업데이트 전 DB에 저장된 상태를 가지고 있다. 한 트랜잭션 내에서 데이터가 업데이트 된 내역이 Entity Manager에 저장 되고, 최종적으로 스냅샷과 비교하여 변경된 내용만 DB에 반영한다.

성능상 이점

지연로딩(lazy-loading)



- 멤버 엔티티는 팀에 속해있고, 주문 엔티티는 주문 정보 엔티티에 속해있는 형태로 DB에 저장되어 있다. Member를 DB에서 조회할 때, 일단 Member의 데이터를 조회하고 나중에 그 Member의 Team 정보를 조회할 수 있다.

남은 문제들

모든게 완벽한가?

| 04

- TypeORM으로 전환 후의 문제점이다.

테스트환경 필요

객체 모델링

querybuilder 방식의 부작용

- > SQL Injection의 위험이 존재..
- > 근데 저희가 못짜서 그런거임...

- 테스트 환경이 필요하다
- 객체 모델링이 필요하다(DTO)
- query builder를 잘못 사용하여 SQL Injection의 위험이 있다.

PORTFOLIO

April.5.2023

THANK YOU!

MY NAME IS

MIRICANVAS

THANK YOU